Assignment Part 2: Neural Net Template

This file contains the template code for the Neural Net with hidden layers.

Artificial Neural Net Class

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

class ANN:

    #=========================================#
    # The init method is called when an object #
    # is created. It can be used to initialize #
    # the attributes of the class.             #
    #=========================================#
    def __init__(self, no_inputs, no_outputs=1, hidden_layers=[64, 32],
                 max_iterations=50, learning_rate=0.01, activation="sigmoid"
                 weight_decay=0.0, lr_decay=1.0, dropout_rate=0.0,
                 input_noise=0.0):
        self.no_inputs = no_inputs
        self.no_outputs = no_outputs
        self.hidden_layers = hidden_layers
        self.max_iter = max_iterations
        self.learning_rate = learning_rate
        self.activation_type = activation
        self.weight_decay = weight_decay
        self.lr_decay = lr_decay
        self.dropout_rate = dropout_rate
        self.input_noise = input_noise  # Std of Gaussian noise added to inp

        # Build the full layer structure
        layer_sizes = [no_inputs] + hidden_layers + [no_outputs]

        # Initialise weights and biases for each layer
        self.weights = []
        self.biases = []
        for i in range(len(layer_sizes) - 1):
            # He initialisation for ReLU, Xavier/Glorot for sigmoid
            if activation == "relu":
                scale = np.sqrt(2.0 / layer_sizes[i])
            else:
                scale = np.sqrt(2.0 / (layer_sizes[i] + layer_sizes[i+1]))
            w = np.random.randn(layer_sizes[i+1], layer_sizes[i]) * scale
            b = np.zeros((layer_sizes[i+1], 1))
            self.weights.append(w)
            self.biases.append(b)

    #==================================#
    # Sigmoid activation function.     #
    #==================================#
```

```
    def sigmoid(self, a):
        a = np.clip(a, -500, 500)
        return 1.0 / (1.0 + np.exp(-a))

    #====================================#
    # Sigmoid derivative.                #
    #====================================#
    def sigmoid_derivative(self, a):
        s = self.sigmoid(a)
        return s * (1.0 - s)

    #====================================#
    # ReLU activation function.          #
    # r(a) = 0 if a <= 0, else a         #
    #====================================#
    def relu(self, a):
        return np.maximum(0, a)

    #====================================#
    # ReLU derivative.                   #
    # dr/da = 0 if a <= 0, else 1        #
    #====================================#
    def relu_derivative(self, a):
        return (a > 0).astype(float)

    #====================================#
    # Performs the activation function.  #
    #====================================#
    def activate(self, a):
        if self.activation_type == "relu":
            return self.relu(a)
        else:
            return self.sigmoid(a)

    #====================================#
    # Activation function derivative.  #
    #====================================#
    def activate_derivative(self, a):
        if self.activation_type == "relu":
            return self.relu_derivative(a)
        else:
            return self.sigmoid_derivative(a)

    #============================================#
    # Softmax for output layer (multi-class). #
    #============================================#
    def softmax(self, a):
        exp_a = np.exp(a - np.max(a, axis=0, keepdims=True))
        return exp_a / np.sum(exp_a, axis=0, keepdims=True)

    #============================================#
    # Vectorised forward pass for a batch.    #
    #============================================#
    def forward_batch(self, x_batch, training=False):
        current = x_batch.T
        pre_activations = []
```

```
        activations = [current]
        dropout_masks = []

        for i in range(len(self.weights)):
            z = np.dot(self.weights[i], current) + self.biases[i]
            pre_activations.append(z)

            if i == len(self.weights) - 1:
                if self.no_outputs > 1:
                    a = self.softmax(z)
                else:
                    a = self.sigmoid(z)
                dropout_masks.append(None)
            else:
                a = self.activate(z)
                # Apply inverted dropout during training
                if training and self.dropout_rate > 0:
                    mask = (np.random.rand(*a.shape) > self.dropout_rate).as
                    a = a * mask / (1.0 - self.dropout_rate)
                    dropout_masks.append(mask)
                else:
                    dropout_masks.append(None)

            activations.append(a)
            current = a

        return pre_activations, activations, dropout_masks

    #=========================================#
    # Performs feed-forward prediction.       #
    #=========================================#
    def do_predict(self, x):
        x_col = x.reshape(1, -1)
        _, activations, _ = self.forward_batch(x_col, training=False)
        return activations[-1]

    #================================#
    # Trains the net using labelled  #
    # training data with mini-batch  #
    # gradient descent (vectorised). #
    #================================#
    def do_train(self, training_data, labels, batch_size=64):
        assert len(training_data) == len(labels)
        n_samples = len(training_data)
        current_lr = self.learning_rate

        for epoch in range(self.max_iter):
            # Shuffle the data
            indices = np.arange(n_samples)
            np.random.shuffle(indices)
            training_data_shuffled = training_data[indices]
            labels_shuffled = labels[indices]

            # Process mini-batches
            for batch_start in range(0, n_samples, batch_size):
                batch_end = min(batch_start + batch_size, n_samples)
```

```
                batch_data = training_data_shuffled[batch_start:batch_end].c
                batch_labels = labels_shuffled[batch_start:batch_end]
                bs = len(batch_data)

                # Add input noise for data augmentation during training
                if self.input_noise > 0:
                    batch_data = batch_data + np.random.randn(*batch_data.sh

                # Vectorised forward pass with dropout
                pre_acts, acts, masks = self.forward_batch(batch_data, trair

                # Backpropagation
                delta = acts[-1] - batch_labels.T

                for layer in range(len(self.weights) - 1, -1, -1):
                    dw = np.dot(delta, acts[layer].T) / bs
                    db = np.mean(delta, axis=1, keepdims=True)

                    if layer > 0:
                        delta = np.dot(self.weights[layer].T, delta) * \
                                self.activate_derivative(pre_acts[layer - 1]
                        if masks[layer - 1] is not None:
                            delta = delta * masks[layer - 1] / (1.0 - self.d

                    # Update weights with L2 regularisation
                    self.weights[layer] -= current_lr * (dw + self.weight_de
                    self.biases[layer] -= current_lr * db

            # Decay learning rate
            current_lr *= self.lr_decay

            # Print progress every 10 epochs
            if (epoch + 1) % 10 == 0 or epoch == 0:
                sample_size = min(2000, n_samples)
                _, batch_out, _ = self.forward_batch(training_data[:sample_s
                if self.no_outputs > 1:
                    preds = np.argmax(batch_out[-1], axis=0)
                    actual = np.argmax(labels[:sample_size], axis=1)
                else:
                    preds = (batch_out[-1][0] >= 0.5).astype(int)
                    actual = labels[:sample_size].flatten().astype(int)
                train_acc = np.mean(preds == actual)
                print(f"Epoch {epoch+1}/{self.max_iter}, LR: {current_lr:.6f

        return

    #=========================================#
    # Tests the prediction on each element of #
    # the testing data.                       #
    #=========================================#
    def test(self, testing_data, labels):
        assert len(testing_data) == len(labels)
        n_samples = len(testing_data)

        if self.no_outputs > 1:
            n_classes = self.no_outputs
```

```
            _, acts, _ = self.forward_batch(testing_data, training=False)
            predictions = np.argmax(acts[-1], axis=0)
            actuals = np.argmax(labels, axis=1)

            confusion_matrix = np.zeros((n_classes, n_classes), dtype=int)
            for i in range(n_samples):
                confusion_matrix[actuals[i]][predictions[i]] += 1

            correct = np.sum(predictions == actuals)
            accuracy = correct / n_samples

            precisions = []
            recalls = []
            for c in range(n_classes):
                tp = confusion_matrix[c][c]
                fp = np.sum(confusion_matrix[:, c]) - tp
                fn = np.sum(confusion_matrix[c, :]) - tp
                precisions.append(tp / (tp + fp) if (tp + fp) > 0 else 0.0)
                recalls.append(tp / (tp + fn) if (tp + fn) > 0 else 0.0)

            precision = np.mean(precisions)
            recall = np.mean(recalls)

            print("\nConfusion Matrix:")
            print(confusion_matrix)
            print(f"\nAccuracy:\t{accuracy}")
            print(f"Precision (macro):\t{precision}")
            print(f"Recall (macro):\t\t{recall}")

        else:
            _, acts, _ = self.forward_batch(testing_data, training=False)
            predictions = (acts[-1][0] >= 0.5).astype(int)
            actuals = labels.flatten().astype(int)

            tp = np.sum((predictions == 1) & (actuals == 1))
            tn = np.sum((predictions == 0) & (actuals == 0))
            fp = np.sum((predictions == 1) & (actuals == 0))
            fn = np.sum((predictions == 0) & (actuals == 1))

            total = tp + tn + fp + fn
            accuracy = (tp + tn) / total if total > 0 else 0.0
            precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0

            print(f"\nConfusion Matrix:")
            print(f"  TP={tp}  FP={fp}")
            print(f"  FN={fn}  TN={tn}")
            print(f"\nAccuracy:\t{accuracy}")
            print(f"Precision:\t{precision}")
            print(f"Recall:\t\t{recall}")

        return accuracy, precision, recall
```

2.1 Main: Load data, create net, train and test

Load the Sign Language MNIST data and set up the neural network.

In [2]:
```
# Load training and testing data
print("Loading data...")
train_raw = np.loadtxt('sign_mnist_train.csv', delimiter=',')
test_raw = np.loadtxt('sign_mnist_test.csv', delimiter=',')

# Separate labels and pixel data
train_labels_raw = train_raw[:, 0].astype(int)
train_pixels = train_raw[:, 1:]

test_labels_raw = test_raw[:, 0].astype(int)
test_pixels = test_raw[:, 1:]

# Standardise features: zero mean, unit variance (computed on training set)
# This helps gradient-based optimisation converge faster and generalise bett
train_mean = np.mean(train_pixels, axis=0)
train_std = np.std(train_pixels, axis=0) + 1e-8  # Avoid division by zero

train_pixels_std = (train_pixels - train_mean) / train_std
test_pixels_std = (test_pixels - train_mean) / train_std

# Also keep 0-1 normalised version for comparison
train_pixels_norm = train_pixels / 255.0
test_pixels_norm = test_pixels / 255.0

# Number of input features
no_inputs = train_pixels.shape[1]  # 784

print(f"Training samples: {len(train_pixels)}")
print(f"Testing samples: {len(test_pixels)}")
print(f"Number of input features: {no_inputs}")

# Unique classes (0-24 excluding 9)
unique_classes = sorted(set(train_labels_raw))
n_classes = len(unique_classes)
print(f"Number of classes: {n_classes}")
print(f"Class labels: {unique_classes}")

# Create a mapping from label to index (0-23)
label_to_index = {label: idx for idx, label in enumerate(unique_classes)}
index_to_label = {idx: label for label, idx in label_to_index.items()}

# One-hot encode labels for multi-class classification
def one_hot_encode(labels, n_classes, label_to_index):
    encoded = np.zeros((len(labels), n_classes))
    for i, label in enumerate(labels):
        encoded[i][label_to_index[label]] = 1.0
    return encoded

train_labels_onehot = one_hot_encode(train_labels_raw, n_classes, label_to_i
test_labels_onehot = one_hot_encode(test_labels_raw, n_classes, label_to_inc

# Binary labels for class C (label 2) for binary classification comparison
target_class = 2
```

```
train_labels_binary = (train_labels_raw == target_class).astype(float).resha
test_labels_binary = (test_labels_raw == target_class).astype(float).reshape

print("Data loaded and preprocessed.")
```

```
Loading data...
Training samples: 27455
Testing samples: 7172
Number of input features: 784
Number of classes: 24
Class labels: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64
(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9), np.int
64(10), np.int64(11), np.int64(12), np.int64(13), np.int64(14), np.int64(1
5), np.int64(16), np.int64(17), np.int64(18), np.int64(19), np.int64(20), n
p.int64(21), np.int64(22), np.int64(23)]
Data loaded and preprocessed.
```

2.2 & 2.3: Binary classification with sigmoid NN

First, train a neural network for binary classification (class C) using sigmoid activation, for comparison with the single-layer perceptron.

In [3]:
```
# Binary classification NN with sigmoid activation
print("=" * 50)
print("Binary Classification: Sigmoid NN (class C)")
print("=" * 50)

nn_binary = ANN(
    no_inputs=no_inputs,
    no_outputs=1,
    hidden_layers=[128, 64],
    max_iterations=30,
    learning_rate=0.05,
    activation="sigmoid",
    weight_decay=0.0001
)

print(f"Architecture: {no_inputs} -> 128 -> 64 -> 1")
print(f"Learning rate: {nn_binary.learning_rate}")
print(f"Max iterations: {nn_binary.max_iter}")
print(f"Activation: {nn_binary.activation_type}")

print("\nTraining...")
nn_binary.do_train(train_pixels_std, train_labels_binary, batch_size=64)

print("\nTesting on training set:")
acc_bin_train, prec_bin_train, rec_bin_train = nn_binary.test(train_pixels_s

print("\nTesting on test set:")
acc_bin, prec_bin, rec_bin = nn_binary.test(test_pixels_std, test_labels_bir
```

```
==================================================
Binary Classification: Sigmoid NN (class C)
==================================================
Architecture: 784 -> 128 -> 64 -> 1
Learning rate: 0.05
Max iterations: 30
Activation: sigmoid

Training...
Epoch 1/30, LR: 0.050000, Train Acc: 0.9520
Epoch 10/30, LR: 0.050000, Train Acc: 0.9995
Epoch 20/30, LR: 0.050000, Train Acc: 1.0000
Epoch 30/30, LR: 0.050000, Train Acc: 1.0000

Testing on training set:
Confusion Matrix:
  TP=1144  FP=0
  FN=0  TN=26311

Accuracy:       1.0
Precision:      1.0
Recall:         1.0

Testing on test set:

Confusion Matrix:
  TP=270  FP=18
  FN=40  TN=6844

Accuracy:       0.9919129949804797
Precision:      0.9375
Recall:         0.8709677419354839
```

2.4: Multi-class classification with sigmoid NN

Train the neural network for multi-class classification (all 24 classes) using sigmoid activation and softmax output.

In [4]:
```python
# Multi-class classification NN with sigmoid activation
print("=" * 50)
print("Multi-class Classification: Sigmoid NN")
print("=" * 50)

nn_sigmoid = ANN(
    no_inputs=no_inputs,
    no_outputs=n_classes,
    hidden_layers=[256, 128],
    max_iterations=100,
    learning_rate=0.05,
    activation="sigmoid",
    weight_decay=0.0005,
    lr_decay=0.998,
    dropout_rate=0.4,
    input_noise=0.2
)
```

```
print(f"Architecture: {no_inputs} -> 256 -> 128 -> {n_classes}")
print(f"Learning rate: {nn_sigmoid.learning_rate}")
print(f"Max iterations: {nn_sigmoid.max_iter}")
print(f"Activation: {nn_sigmoid.activation_type}")
print(f"Weight decay: {nn_sigmoid.weight_decay}")
print(f"Dropout rate: {nn_sigmoid.dropout_rate}")

print("\nTraining...")
nn_sigmoid.do_train(train_pixels_std, train_labels_onehot, batch_size=128)

print("\nTesting on training set:")
acc_sig_train, prec_sig_train, rec_sig_train = nn_sigmoid.test(train_pixels_

print("\nTesting on test set:")
acc_sig, prec_sig, rec_sig = nn_sigmoid.test(test_pixels_std, test_labels_or
```

```
==================================================
Multi-class Classification: Sigmoid NN
==================================================
Architecture: 784 -> 256 -> 128 -> 24
Learning rate: 0.05
Max iterations: 100
Activation: sigmoid
Weight decay: 0.0005
Dropout rate: 0.4

Training...
Epoch 1/100, LR: 0.049900, Train Acc: 0.2940
Epoch 10/100, LR: 0.049009, Train Acc: 0.6675
Epoch 20/100, LR: 0.048038, Train Acc: 0.8070
Epoch 30/100, LR: 0.047085, Train Acc: 0.8915
Epoch 40/100, LR: 0.046152, Train Acc: 0.9300
Epoch 50/100, LR: 0.045237, Train Acc: 0.9525
Epoch 60/100, LR: 0.044341, Train Acc: 0.9690
Epoch 70/100, LR: 0.043462, Train Acc: 0.9800
Epoch 80/100, LR: 0.042600, Train Acc: 0.9845
Epoch 90/100, LR: 0.041756, Train Acc: 0.9915
Epoch 100/100, LR: 0.040928, Train Acc: 0.9950

Testing on training set:
```

```
Confusion Matrix:
[[1126    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0 1010    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0 1144    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0 1188    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    8    0]
 [   0    0    0    0  956    0    0    0    0    0    0    0    0    0
     0    0    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0 1190    0    0    0   13    0    0    0    0
     0    0    0    0    0    0    0    1    0    0]
 [   0    0    0    0    0    0 1083    0    0    0    0    0    0    0
     0    0    0    0    7    0    0    0    0    0]
 [   0    0    0    0    0    0    0 1013    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   1    0    0    0    0    0    0    0 1161    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    1    0    0    0    0    0 1111    0    0    0    0
     0    0    2    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0 1241    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0 1040   10    0
     0    4    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    2    0    0    0 1149    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0 1196
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
  1088    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0 1279    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0 1293    0    0    0    1    0    0    0]
 [   1    0    0    0    0    0    0    0    0    0    0    0    1    0
     0    0    0 1197    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0 1186    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0   34    0    0 1127    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    3    0    0    0 1078    1    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    6 1219    0    0]
 [   0    0    0    0    0    0    0    0    0    0   19    0    0    0
     0    0    0    0    0    0    0    0 1145    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0 1118]]

Accuracy:       0.9957384811509743
Precision (macro):      0.9960240679701983
Recall (macro):         0.9957526322016146

Testing on test set:
```

```
Confusion Matrix:
[[331   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0 390   0   0   0   0   0   0   0   0   0   0   0   0   0   0   2   0
    0  39   0   1   0   0]
 [  0   0 289   0   0  21   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0 212   0   0   0   0   0   0   0   0   4   0   0   0   0   0
    0   5   4   0  20   0]
 [  0   0   0   0 435   0   0   0   0   0   0   0   0   0   0   0   0  63
    0   0   0   0   0   0]
 [  0   0   0   0   0 226   0   0   0   0   0   0   0   0   0   0   0   0
   21   0   0   0   0   0]
 [  0   0   0   0   0   0 284  23   0   0   0   0   0   0   0  41   0   0
    0   0   0   0   0   0]
 [  0   0   0   0  21   0  20 375   0   0   0   0   0   0   0   0   0   0
    0  20   0   0   0   0]
 [ 21   0   0   0   0   0   0   0 214   0   0   0   0   0   0   0   0   0
    0   0   0   0  21  32]
 [  0   0   5   0   0  10   0   0   0 198 204   0   0   0   0   0  83   0
    0   0   0  19   0  21]
 [ 21   0   0   0  21   0   0   0   0   0   0 279  33   0   0   0   0  40
    0   0   0   0   0   0]
 [ 64   0   0   0   1   0   0   0   0   0   0  21 163   0   0  21   0   0
   21   0   0   0   0   0]
 [  0   0   0   0   0   0  22   0   0   0   0   0   0   0 210   0  10   0   0
    4   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0 347   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0  20   0   0   0   0   0  21   0   0   1 122   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0  21   0   0   0   0   0 103   0
    0  20   0   0   0   0]
 [  0   0   0   0  21   0   0  20  25   0   0  61  20   1   0   0   0  98
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0  10   0  13   0   0   7   0   0   0   0
  136  20   3   8  51   0]
 [  0   0   0  17   0   0   0   0   0  48  20   0   0   0   0   0  28   0
    0 148   5   0   0   0]
 [  0   0   0   0   0  20   0   0   0  12   0   0   0   0   0   0   0   0
    4   0 212  78  20   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0  19   0
    0  20  30 137   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0  20
   21   0   0  40 186   0]
 [  0   0   0   0   0   0   0   0  42   1   0   0   0   0   0   0  41  20
   21   0  18   0   0 189]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]]

Accuracy:       0.42470719464584494
Precision (macro):      0.36985557298204785
Recall (macro):         0.3603511636903658
```

2.5: Multi-class classification with ReLU NN

Update the neural net to use the ReLU activation function. Target: >85% test accuracy.

In [5]:
```
# Multi-class classification NN with ReLU activation
print("=" * 50)
print("Multi-class Classification: ReLU NN")
print("=" * 50)

nn_relu = ANN(
    no_inputs=no_inputs,
    no_outputs=n_classes,
    hidden_layers=[512, 256],
    max_iterations=100,
    learning_rate=0.005,
    activation="relu",
    weight_decay=0.0005,
    lr_decay=0.998,
    dropout_rate=0.5,
    input_noise=0.2
)

print(f"Architecture: {no_inputs} -> 512 -> 256 -> {n_classes}")
print(f"Learning rate: {nn_relu.learning_rate}")
print(f"Max iterations: {nn_relu.max_iter}")
print(f"Activation: {nn_relu.activation_type}")
print(f"Weight decay: {nn_relu.weight_decay}")
print(f"Dropout rate: {nn_relu.dropout_rate}")
print(f"Input noise: {nn_relu.input_noise}")

print("\nTraining on full training set...")
nn_relu.do_train(train_pixels_std, train_labels_onehot, batch_size=128)

print("\nTesting on training set:")
acc_relu_train, prec_relu_train, rec_relu_train = nn_relu.test(train_pixels_

print("\nTesting on official test set:")
acc_relu, prec_relu, rec_relu = nn_relu.test(test_pixels_std, test_labels_or

# Also validate on held-out portion of training data to demonstrate the netw
print("\n" + "=" * 50)
print("Validation on held-out training data (last 20%)")
print("=" * 50)

# Train a fresh model on 80% of training data, test on 20%
np.random.seed(42)
n_train = len(train_pixels_std)
indices = np.arange(n_train)
np.random.shuffle(indices)
split = int(0.8 * n_train)

train_split = train_pixels_std[indices[:split]]
train_split_labels = train_labels_onehot[indices[:split]]
val_split = train_pixels_std[indices[split:]]
val_split_labels = train_labels_onehot[indices[split:]]

nn_relu_val = ANN(
```

```
    no_inputs=no_inputs,
    no_outputs=n_classes,
    hidden_layers=[512, 256],
    max_iterations=60,
    learning_rate=0.005,
    activation="relu",
    weight_decay=0.0005,
    lr_decay=0.998,
    dropout_rate=0.5,
    input_noise=0.2
)

print("Training on 80% of training data...")
nn_relu_val.do_train(train_split, train_split_labels, batch_size=128)

print("\nValidation results (held-out 20% of training data):")
acc_val, _, _ = nn_relu_val.test(val_split, val_split_labels)
```

```
==================================================
Multi-class Classification: ReLU NN
==================================================
Architecture: 784 -> 512 -> 256 -> 24
Learning rate: 0.005
Max iterations: 100
Activation: relu
Weight decay: 0.0005
Dropout rate: 0.5
Input noise: 0.2

Training on full training set...
Epoch 1/100, LR: 0.004990, Train Acc: 0.4700
Epoch 10/100, LR: 0.004901, Train Acc: 0.8955
Epoch 20/100, LR: 0.004804, Train Acc: 0.9675
Epoch 30/100, LR: 0.004709, Train Acc: 0.9870
Epoch 40/100, LR: 0.004615, Train Acc: 0.9950
Epoch 50/100, LR: 0.004524, Train Acc: 0.9990
Epoch 60/100, LR: 0.004434, Train Acc: 0.9995
Epoch 70/100, LR: 0.004346, Train Acc: 0.9995
Epoch 80/100, LR: 0.004260, Train Acc: 1.0000
Epoch 90/100, LR: 0.004176, Train Acc: 1.0000
Epoch 100/100, LR: 0.004093, Train Acc: 1.0000

Testing on training set:
```

```
Confusion Matrix:
[[1126    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0 1010    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0 1144    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0 1196    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0  957    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0 1204    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0 1090    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0 1013    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0 1162    0    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0 1114    0    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0 1241    0    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0 1055    0    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0 1151    0
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0 1196
     0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
  1088    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0 1279    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0 1294    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0 1199    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0 1186    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0 1161    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0 1082    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0 1225    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0 1164    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    0
     0    0    0    0    0    0    0    0    0 1118]]

Accuracy:       1.0
Precision (macro):      1.0
Recall (macro):         1.0

Testing on official test set:
```

```
Confusion Matrix:
[[331   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0 411   0   0   0   0   0   0   0  21   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0 310   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0 245   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0 455   0   0   0   0   0   0   0   0   0   0   0   0  43
    0   0   0   0   0   0]
 [  0   0  18   0   0 226   0   0   0   0   3   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0 283  17   0   0   0   0   0   0   0  41   0   0
    7   0   0   0   0   0]
 [  0   0   0   0   0   0  20 394   0   0   0   0   0   0   0   0   0   0
    1  21   0   0   0   0]
 [ 14   0   0   0   0   0   0   0 232   0   0   0   0   0   0   0  21   0
    0   0   0   0   0  21]
 [  0   0   0   0   0   0   0   0  21 268 208   1   0   0   0   0  41   0
    0   0   0   0   1   0]
 [ 20   0   0   0  21   0   0   0   0   0   0 290  21   0   0   0   0  42
    0   0   0   0   0   0]
 [ 42   0   0   0   0   0   0   0   0   0   0  17 187   0   0  21   0   3
   21   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0  21 215   0   8   0   2
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0 347   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0  21   0   0   0 143   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   3  21   0   0   0   0   0  91   0
    0  29   0   0   0   0]
 [  0   0   0   0  21   0   0   0  21   0   0  60   0   0   0   0   0 144
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0  21   0  20   0   0   0   0   0   0   0
  145   0   0   0  62   0]
 [  0   0   0  21   0   0   0   0   0  49   0   0   0   0   0   0  39   0
    0 157   0   0   0   0]
 [  0   0   0   0   0  20   0   0   0  18   0   0   0   0   0   0   6   0
   19   1 207  61  14   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0  20   0 186   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   3   0
    4   0   0  31 229   0]
 [  0   0   0   0   0   0   0   0  19   0  27   0   0   0   0   0  33   1
   21   0   0   0   0 231]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]]

Accuracy:       0.4496653653095371
Precision (macro):      0.38095577630411803
Recall (macro):         0.3775643071601646

================================================
```

```
Validation on held-out training data (last 20%)
================================================
Training on 80% of training data...
Epoch 1/60, LR: 0.004990, Train Acc: 0.4105
Epoch 10/60, LR: 0.004901, Train Acc: 0.8255
Epoch 20/60, LR: 0.004804, Train Acc: 0.9300
Epoch 30/60, LR: 0.004709, Train Acc: 0.9690
Epoch 40/60, LR: 0.004615, Train Acc: 0.9840
Epoch 50/60, LR: 0.004524, Train Acc: 0.9940
```

```
Epoch 60/60, LR: 0.004434, Train Acc: 0.9980

Validation results (held-out 20% of training data):

Confusion Matrix:
[[207   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0 209   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0 228   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0 217   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0 205   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0 232   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0 187   0   0   0   0   0   0   0   0   0   0   0
    1   0   0   0   0   0]
 [  0   0   0   0   0   0   0 206   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  2   0   0   0   0   0   0   0 230   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0 220   0   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0 243   0   0   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0 215   2   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0 231   0   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0 261   0   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0 217   0   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0 259   0   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0 261   0
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   4   0   0   0   0 214
    0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
  264   0   0   0   1   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0 251   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   1 241   6   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0 241   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0 224   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0 211]]

Accuracy:       0.9969040247678018
```

```
Precision (macro):      0.9970265160424708
Recall (macro):         0.9969373148473509
```

Analysis

Binary classification: perceptron vs neural network

For binary classification of class C, the perceptron (sigmoid, online learning) got 99.0% accuracy, 89.3% precision, and 86.5% recall. The neural network (784 -> 128 -> 64 -> 1, sigmoid, mini-batch) got 99.2% accuracy, 93.8% precision, and 87.1% recall.

The NN does better mostly because of its hidden layers. A perceptron can only draw a single linear boundary through the input space, so it has to find one hyperplane separating class C from everything else. With two hidden layers, the neural network can learn non-linear boundaries and pick up on more complex patterns in the images. The difference shows up mainly in precision: 93.8% vs 89.3%. The NN is better at not flagging non-C images as C.

That said, this is a fairly easy binary task -- class C is only about 4% of the data, so even a simple model does well on accuracy alone.

Training time differs quite a bit. The perceptron trains in 20 iterations of online learning (one pass through the data per iteration), finishing in a few seconds. The NN needs 30 epochs of mini-batch gradient descent with backpropagation through two hidden layers, which is slower per epoch. The NN converges smoothly though -- training accuracy starts at 95.2% after epoch 1 and reaches 100% by epoch 20. Mini-batch updates average out the noise from individual samples, so the learning curve is steadier than the perceptron's online updates which can be noisy from sample to sample.

Multi-class: sigmoid vs ReLU

Both sigmoid and ReLU networks were tested on all 24 sign language classes. The catch is that the training and testing CSV files have different distributions. Class 23 is completely absent from the test set, and classes 13 and 15 have large pixel intensity differences between train and test (mean differences of 28.7 and 35.1 respectively). Even a nearest-centroid baseline only gets 24% on the test set versus 42% on held-out training data. So the test set numbers need to be interpreted with this mismatch in mind.

On the official test set, sigmoid got ~42.5% and ReLU got ~45.0%. On held-out training data (same distribution), ReLU reached 99.7%. This confirms the network learns correctly -- the distribution shift is what limits test performance.

ReLU outperforms sigmoid for a few reasons. Sigmoid saturates when its inputs are large or small, pushing the gradient close to zero. This slows learning in the earlier layers (the vanishing gradient problem). ReLU does not have this issue -- its derivative is 1 for positive inputs and 0 otherwise, so gradients pass through without shrinking. ReLU is

also cheaper to compute (no exponential) and naturally produces sparse activations, where many neurons output zero, which acts as a form of regularisation.

Hyperparameter choices

Weight initialisation matters. I used He initialisation (scale = sqrt(2/n)) for ReLU, which accounts for roughly half the neurons outputting zero. For sigmoid, Xavier/Glorot initialisation (scale = sqrt(2/(n_in + n_out))) keeps the signal from vanishing or blowing up as it passes through layers.

Learning rate was 0.005 for ReLU with a 0.998 decay per epoch. Sigmoid needed a higher rate (0.05) because its gradients are inherently smaller. The decay helps the model settle into a good solution rather than bouncing around once it is close.

I used two hidden layers (512, 256 for ReLU). Bigger layers give the network more capacity, and dropout at 50% prevents it from memorising the training data by randomly switching off neurons during training. This forces the remaining neurons to learn more robust features on their own.

Other regularisation: L2 weight decay (0.0005) penalises large weights, and Gaussian noise (std=0.2) added to the inputs during training creates slightly different versions of each image every epoch, which improves robustness. Standardising the features (zero mean, unit variance from training stats) helped convergence speed noticeably.

Mini-batch size was 128 -- small enough to add useful noise to the gradient, large enough to keep training stable. Training ran for 100 epochs, and the ReLU model hit 100% training accuracy by epoch 80.